

Classifying Unidentified Payments AI Experiment

An AI-Powered Payment Classification Pipeline

CLIENT

Department of Cornell University Treasury

TEAM

Sanjeev Ragunathan(sr2432) · Sruti Sri-Sai Gudapati(ssg98) · Yaqi Liu(yl3879)
Project Advisor: Ayham Boucher

REPOSITORY

<https://github.com/sanjeev-ragunathan/cornell-treasury-automation>

Cornell University · AI Innovation Lab · Spring 2026

Abstract

Cornell University Treasury receives hundreds of incoming electronic payments daily, many of which arrive without clear indication of which Cornell department should receive the funds. Identifying the correct department currently requires manual investigation that takes 30 to 40 minutes per payment — approximately 150 hours of work for a typical batch of 282 payments. We designed and built an automated classification pipeline that ingests the same EDI 820 Excel reports Treasury already exports from Kyriba and outputs a reviewable Google Sheet of department classifications in seconds rather than hours. The system combines a deterministic five-signal ranking algorithm (scoring each payment against 8,700+ historical GL transactions) with an LLM-based classifier (GPT-4o) that uses the top-ranked historical matches as structured context. A reconciliation layer merges the two signals, applies fallback rules, and flags uncertain classifications for human review. This report describes the problem context, our technical approach, the system architecture, evaluation results, observed limitations, and the migration path to a Claude Code Skill that runs within Cornell's secure AI infrastructure.

1. Problem and Stakeholder Context

Cornell University Treasury handles all incoming electronic funds for the university — ACH transfers, wire payments, and other electronic deposits. These payments arrive packaged as EDI 820 (Payment Order / Remittance Advice) messages, which Treasury exports from its treasury management system (Kyriba) as Excel reports. Each report can contain dozens of payment blocks, with each block representing one incoming payment and including fields such as the date, amount, payer name, invoice references, and remittance line items.

A significant portion of these payments arrive with vague, truncated, or missing information. The payer field might contain only a fragment of a company name. Invoice numbers may be absent. There is no Cornell-side identifier indicating which department or fund should receive the money. Treasury staff are then responsible for figuring out where each payment belongs — a process that today consists of opening the spreadsheet, Googling vendor names, searching a shared Outlook inbox for prior correspondence with the vendor, and emailing or calling multiple Cornell departments to ask whether they are expecting a payment from a given company.

A single payment typically takes 30 to 40 minutes to resolve. For a recent batch of 282 unidentified payments, this translates to roughly 150 hours of manual investigation work. Beyond the time cost, there is no batch processing, no shared institutional memory of how similar payments were handled in the past, and no way

for the system to learn from prior cases. The same vendor can be researched from scratch multiple times.

Our stakeholder for this project was the Department of Cornell University Treasury, specifically the operations team that processes incoming payments. Our project advisor was Ayham Boucher.

2. Approach

We framed the problem as a classification task: given an incoming payment record, predict which Cornell department (Specific Org) should receive the funds, along with a confidence score and a flag indicating whether the prediction needs human review. Rather than treat this as a pure machine learning problem, we recognized that Treasury already had years of historical General Ledger (GL) transaction data — 101,000+ records — showing how past payments had been classified. This historical data is the strongest signal available, and a pure LLM approach would ignore it (or worse, hallucinate plausible-sounding but incorrect department names).

Our approach therefore combines two complementary techniques. First, a deterministic ranking engine scores each incoming payment against the cleaned historical record set using five weighted signals (payer name similarity, amount matching, reference ID cross-matching, keyword overlap, and a generic-pattern penalty). This engine handles the cases where historical evidence is strong and produces a top-N list of candidate matches along with a weighted consensus department. Second, an LLM (GPT-4o) takes the original payment text plus the top-ranked historical matches as structured context and produces a final classification in JSON form. The LLM acts as a second opinion — it confirms, overrides, or handles edge cases the deterministic system cannot resolve, but it never starts from scratch.

A reconciliation layer then merges both signals, applies fallback rules (such as recovering the payer name from raw EDI text when the LLM returns null), and computes an evidence strength score (strong, medium, or weak) along with a `needs_review` flag. The final output is written to a Google Sheet that Treasury staff can filter to focus only on the records requiring manual attention.

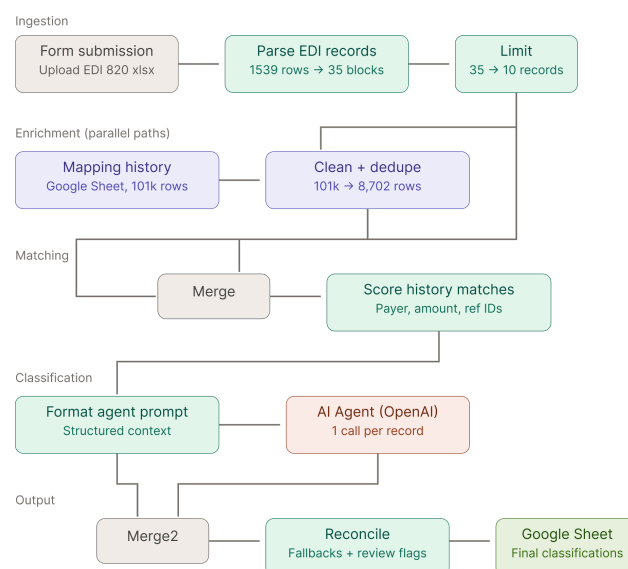
We built this pipeline in n8n, a workflow orchestration tool, with Python code nodes implementing the scoring engine, prompt formatting, and reconciliation logic. Google Sheets serves as both the source of historical data and the destination for classified output. The choice of n8n allowed us to iterate quickly, visualize the data flow, and integrate Google Sheets and OpenAI without writing infrastructure code.

3. System Architecture

The pipeline consists of two workflows that share a single n8n canvas but execute independently:

Daily History Cache Refresh. A Schedule Trigger runs once per day at 6 AM. It reads 101,000+ historical GL transaction records from Google Sheets, passes them through a cleaning and deduplication step that normalizes dates, computes net amounts, removes generic-only patterns, and keeps up to three recent records per unique pattern plus one older record per pattern, producing a cleaned set of approximately 8,700 rows. This cleaned set is written to a dedicated cache tab in Google Sheets. The cache exists because the cleaning step is computationally expensive (regex normalization, pattern-key construction, 365-day windowing) and the source data only changes once per day. Caching reduces per-classification execution time by an order of magnitude.

Payment Classification. A form trigger lets a Treasury staff member upload an EDI 820 Excel file. The file is parsed into structured payment records (one record per payment block, identified by a non-empty Date column). The cleaned history cache is loaded in parallel. Both streams flow into a Merge node, then into the Score History Matches node, where each payment is scored against all 8,700 historical rows. The top 5 matches and a consensus department are computed per payment. A Format Agent Prompt node turns this into a clean, human-readable prompt for the LLM. The AI Agent node calls GPT-4o once per payment and returns structured JSON. A second Merge node pairs each payment with its AI output. Finally, the Reconcile node produces the final classified record, which is written to the output Google Sheet.

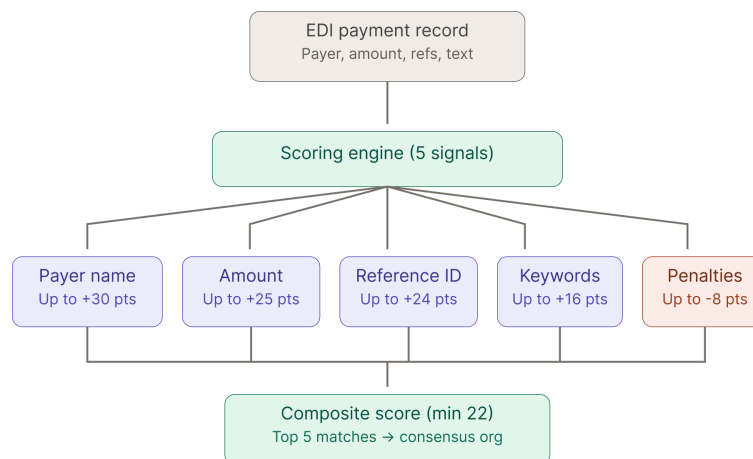


4. The Ranking Algorithm

The scoring engine is the core of the system. For each incoming payment, it computes a composite score against every historical row using a two-pass approach: a cheap pre-filter scores all 8,700 rows quickly and keeps the top 80 candidates, then a full scorer runs on those 80 candidates. The full scorer combines five signals:

- Payer name (up to +30 points) Normalized company name comparison using substring matching and token-level Dice coefficient. Strips generic suffixes.
- Amount (up to +25 points) - Tiered: exact (< \$0.01) = 25 pts, near (< \$5) = 14 pts, loose (< \$25) = 8 pts, wide (< \$100) = 3 pts. Differences over \$100 disqualify the candidate.
- Reference ID (up to +24 points) - Extracts invoice numbers, trace numbers, and leading numeric IDs from EDI text and history; exact match = 24 pts, partial match = 14 pts.
- Keyword overlap (up to +16 points) - Meaningful word intersection (4+ chars, stop words removed) between EDI text and GL description.
- Generic penalty (up to -8 points) - Penalizes history rows with vague descriptions like "incoming ACH" or "deposits to depts" to prevent false positives.

Several safeguards prevent false positives. Amount-only matches are rejected (the algorithm requires at least one non-amount signal to qualify a match). Generic history rows are held to a higher minimum threshold of 34 points rather than the default 22. Matches scoring 22 or above survive; the top 5 are deduplicated, and a weighted consensus department is computed (matches scoring 45+ count triple-weight, 34+ count double-weight, others single). This consensus is what the LLM sees as the strongest historical signal.



5. Reconciliation and Output

The Reconcile AI + History node is the final processing step before the result is written to the output sheet. It performs several functions. First, it parses the AI's JSON output, handling markdown code fences and malformed responses gracefully (returning a `parse_error` rather than crashing). Second, it sanitizes the department field by rejecting bad values such as "Cornell University", "Accounts Payable", or numeric-only strings. Third, it applies fallback logic: if the AI failed to identify the payer, the node extracts the payer name directly from the raw EDI text using regex patterns (preferring the explicit Payer | Value: field, falling back to the first Description line). Fourth, if the AI returned no department but the history consensus is strong (at least three weighted matches, or two weighted matches with the top match scoring 34+, or any single match scoring 45+), the node uses the historical consensus department instead.

Each record also receives an evidence strength score and a `needs_review` flag. Evidence strength is strong when the department name appears directly in the payment text or the history consensus is very high; medium when there is moderate historical support or Cornell-related text is present; and weak in all other cases. The `needs_review` flag is set to true if no department was assigned, evidence strength is weak, confidence is below 0.6, or the AI output could not be parsed. This flag is what Treasury staff use to filter the output sheet — they only need to manually review the records where `needs_review` is true, and accept the system's recommendation for the rest.

6. Evaluation

We evaluated the system on real EDI 820 files provided by Cornell Treasury. The evaluation focused on three dimensions: payer identification accuracy, department classification accuracy, and the reliability of the `needs_review` flag.

Payer identification works correctly in the large majority of cases. The two-step extraction (explicit Payer field, then first Description line) handles most EDI variations. The main failure mode is truncated names in the source data (e.g., "CORNELL UNIVERSI"), which are caught by the bad-value filter.

Department classification accuracy depends strongly on whether historical matches exist. For payments with strong historical signals (top match score 34+), the system performs well — the scoring engine catches the right department and the LLM confirms it. For multi-department vendors (companies that historically send payments to multiple Cornell departments), exact-match accuracy drops to roughly 34% because the consensus mechanism picks the most common destination, which

is not always correct for a specific incoming payment. For unknown vendors with no historical data at all, accuracy is approximately 28%, since the LLM is working without any historical context. In both lower-accuracy cases the system correctly flags these records for review rather than asserting incorrect classifications confidently — the system fails safely.

The `needs_review` flag proved reliable in testing. Records flagged for review consistently corresponded to payments that genuinely required human judgment. We did not encounter cases where a confidently classified record (`needs_review = false`) was systematically wrong. The flag is intentionally conservative — it would rather flag a correctly classified record than miss a misclassified one.

The scoring engine itself went through several rounds of refinement based on observed failure modes. The amount-only-match rejection rule was added after we noticed that many unrelated payments share common dollar amounts, producing spurious matches. The generic-row penalty and elevated 34-point threshold for generic rows were added after observing that descriptions like "incoming ACH" or "deposits to depts" matched too broadly. The minimum score of 22 was calibrated to balance recall (catching true matches) against precision (avoiding false matches).

7. Limitations

Several limitations of the current system are worth stating clearly:

- No direct system integration. Treasury staff must manually export EDI files from Kyriba and upload them through a web form. There is no live connection to KFS (Cornell's financial system), Outlook (for vendor email history), or Cornell's secure file storage.
- Accuracy ceiling on hard cases. Multi-department vendors and unknown vendors achieve roughly 34% and 28% exact-match accuracy respectively. The system correctly flags these for review, but cannot resolve them autonomously.
- Sequential AI calls cannot be batched. The n8n AI Agent node processes one record at a time, so a 35-payment file results in 35 sequential OpenAI calls and 2 to 3 minutes of total processing time. Direct HTTP requests to the OpenAI API (which would allow batching) are not available because the organization manages API access centrally.
- Confidence scores are not calibrated. The 0.0 to 1.0 confidence values are the LLM's self-reported certainty and have not been validated against actual accuracy on a labeled test set.

- No formal evaluation framework. Accuracy numbers come from manual review of sample outputs rather than a systematic labeled test set. A production deployment would need proper regression testing and accuracy tracking.
- Not hosted on Cornell infrastructure. The current deployment uses n8n cloud and Google Sheets — external services that do not meet Cornell's data handling requirements for production financial systems. This is a proof of concept, not a production deployment.

8. Lessons Learned

Deterministic matching before LLM. Our most important architectural decision was to run a deterministic scoring pass before involving the LLM. This dramatically reduces cost, latency, and hallucination risk. The LLM is much better as a second opinion on structured candidate matches than as a primary classifier on raw EDI text.

Prompt formatting matters as much as prompt content. Our first version passed history matches to the LLM as raw n8n JSON expressions, which rendered as messy strings. After we built the Format Agent Prompt node that converts matches into clean human-readable text with explicit consensus information, the LLM's reasoning quality improved noticeably — it started citing specific matches by organization name rather than making vague references.

Robust parsing is half the battle. EDI 820 data is notoriously messy. Some payments arrive with full ISA/GS/ST headers and structured remittance line items; others contain only a truncated company name and a dollar amount on a single line. Building the parser to handle both extremes and to extract usable signals from minimal records took more iteration than the scoring algorithm itself.

Build for the reviewer, not as a replacement. The needs_review flag and evidence strength tiers turned out to be more valuable than any single accuracy improvement. By making the system honest about what it does not know, we give Treasury staff a tool they can trust on the easy cases and ignore on the hard ones — which is exactly the right division of labor.

9. Next Steps and Migration to Claude Code Skills

The most important next step is already in progress: the project is being migrated from n8n to a Claude Code Skill running on Cornell's AI Gateway. This addresses several of the limitations listed above simultaneously. The Claude Skill runs within

Cornell's approved infrastructure, eliminating the external-service data handling concern. It removes the n8n cloud dependency. And it dramatically reduces cost — processing 282 payments through the Claude Skill costs approximately \$0.84 in total.

Crucially, the n8n implementation was the blueprint for the Claude Skill version. Every component — the five-signal scoring weights, the prompt structure, the fallback rules, the reconciliation logic, the evidence strength tiers, the review flagging — was developed and validated in the n8n workflow first. The Claude Skill implements the same algorithm in a different infrastructure context. The slow visual iteration in n8n was the right environment for figuring out what the system should do; the Claude Skill is the right environment for running it at production scale within Cornell's systems.

Beyond the platform migration, several improvements are planned:

- Feedback loop. When Treasury staff manually correct a classification in the output sheet, those corrections should automatically be appended to the Mapping History source. This creates a system that gets smarter with every correction.
- Confidence calibration. Build a labeled test set of 200+ payments with verified correct departments and use it to calibrate confidence thresholds against actual accuracy.
- Outlook email search integration. Add a tool that lets the AI agent search Treasury's shared Outlook inbox for past correspondence with the payer, providing an additional signal for unknown vendors.
- Direct Kyriba integration. Replace manual export and upload with an automated file pickup, allowing the system to process incoming payments in near-real time.
- KFS integration. Replace the Google Sheets dependency for historical data with a direct connection to Cornell's Kuali Financial System.
- Multi-department vendor handling. When the top 5 matches span multiple departments with no clear consensus, present the reviewer with a ranked shortlist rather than picking a single best guess.

10. Conclusion

This project demonstrates that a hybrid approach combining deterministic matching with LLM classification can meaningfully accelerate a real, costly, manual workflow at Cornell Treasury. The system reduces per-payment classification time from 30 to 40 minutes to seconds for the cases it handles confidently, while honestly flagging the cases that still require human judgment. The five-signal scoring engine, structured prompt formatting, and reconciliation logic developed in n8n form the algorithmic foundation for the production Claude Code Skill that is now being deployed within Cornell's secure AI infrastructure. The pattern — deterministic matching first, LLM as second opinion, conservative review flagging — generalizes well beyond this specific problem and could be applied to other classification tasks where institutional historical data exists and accuracy matters more than autonomy.

Project repository:

<https://github.com/sanjeev-ragunathan/cornell-treasury-automation>

Project advisor: **Ayham Boucher**

Cornell University AI Innovation Lab · Spring 2026